Course Project

Semantic Web Integration of Empirical Thesis Metadata

Grigoris Markakis, Maria Katsamani, George Mpitzes

Abstract

Even though undergraduate and postgraduate theses, including **Bachelor, Master, and Doctoral** projects, contain valuable empirical data, they often end up as "dark data" in University Repositories. This happens because current systems use simple standards like Dublin Core, which are not detailed enough to describe critical empirical components, such as methodologies, sampling sizes, and sampling strategies. The main goal of this project is to fix this problem by turning a core XML schema into a well-formed **OWL ontology**. By mapping 33 different metadata fields to global standards like **Dublin Core Terms** and **Schema.org**, we want to make this data compatible with the rest of the world and much easier for computers to read. To validate this framework, a **Knowledge Graph** was constructed using data extracted from 31 real thesis samples containing empirical research. The resulting triples were deployed in a **Virtuoso triple store**, providing a robust environment for semantic queries. Finally, the system's utility was demonstrated through a **SPARQL-based web application**, which allows users to find specific research details that are usually hidden in traditional repository systems.

Keywords: *Semantic Web, Empirical Research, Student Theses, Metadata Standards, Dublin Core, Schema.org, OWL ontology, Knowledge Graph, SPARQL, Virtuoso, Gray Literature.*

1. Introduction

Institutional repositories are especially important for saving student research, but they often face a significant limitation: they do not have structured metadata for empirical studies. Most repositories today use simple standards like Dublin Core, which are good for general information but too simple to describe complex research details like methodologies or sampling strategies. Because of this, many valuable details from undergraduate and postgraduate theses remain "dark data": research that exists in the system but is difficult for other researchers to find or query.

The main goal of this project is to fix this problem by transforming a core XML schema into a formal OWL ontology. By mapping 33 specific metadata fields to well-known vocabularies like Dublin Core Terms and Schema.org, we want to move past simple descriptions and build a real Knowledge Graph. This semantic approach helps us represent complex connections in the data, making empirical research in the social sciences much easier to discover and use through modern web technologies.

2. System Architecture and Implementation

2.1 Ontology Design

2.1.1 The Core Ontology Structure

Logical Separation of Concepts: Instead of keeping all data in one long list, we organized the ontology into five primary disjoint classes:

- **Thesis**: The central entity representing the thesis itself.
- **Writer**: Representing the author of the research.
- **Survey**: Capturing the empirical research component.
- **Sample**: Describing the methodology and units of study.
- **Spacetime**: Defining the geographical and temporal context.

This ensures the system maintains logical integrity; for example, a "Writer" (a person) cannot accidentally be classified as a "Survey" (a research component).

Hierarchical Intelligence: By creating a **Thesis** class with specific sub-classes for **Bachelors, Master, and Doctoral** levels, we enabled the system to understand relationships. This allows a SPARQL query for "Thesis" to automatically include all specific degree levels without manual filtering. Similarly, we organized the **Survey** class into three sub-sections: **Qualitative, Quantitative, and Mixed**.

Creating Semantic Relationships: We introduced **Object Properties,** such as *hasWriter*, *hasSurvey* and *hasSample,* to act as the "connective tissue" of our graph. In the original XML, these elements were just positioned near each other; in our OWL ontology, they are formal logical relationships that a computer can follow to discover context.

2.1.2 Transition from XML to OWL

While the initial project requirements were based on **33 fields** from the XML schema, our finalized ontology contains a higher number of declarations. This increase was necessary to transform a flat data structure into a functional **Knowledge Graph**. Specifically, we expanded the model for two main reasons:

1. Hierarchical Intelligence

2. Semantic Relationships

In summary, we preserved all **33 original concepts** , but the OWL environment required extra properties and classes to ensure the system is fully searchable via **SPARQL**.

2.1.3 Semantic Mapping to Global Standards

To ensure our local project could "speak" to the rest of the world, we took specific actions to map our 33 metadata fields to international standards like **Dublin Core** and **Schema.org**.

- **Semantic Alignment**: We used the rdfs:seeAlso and rdfs:isDefinedBy properties within Protégé to provide a "semantic bridge".
- **Standardization Examples**: For instance, we mapped our local *institution* field to *dcterms:publisher* and *schema:sourceOrganization*. This means our data is no longer isolated, it is globally compatible, allowing external systems to understand our research components without manual translation.

Table 1. *Metadata Crosswalk between the OWL Ontology, Dublin Core, and Schema.org (Representative Subset).*

OWL	**Data Property**	**Dublin Core**	**Schema.org**
Author Identity	firstName, surName	dcterms:creator	schema:givenName, schema:familyName
	gender	—	schema:gender
	nameOfSupervisor	dcterms:contributor	schema:contributor
Institutional Context	institution	dcterms:publisher	schema:sourceOrganization
	department	—	schema:department
Empirical Metrics	sampleDescription	dcterms:description	schema:description
	spatialCoverage	dcterms:spatial	schema:spatialCoverage
	population	dcterms:coverage	schema:populationType
Discoverability	keyWords	dcterms:subject	schema:keywords
	language	dcterms:language	schema:inLanguage
	libraryLink	dcterms:identifier	schema:url

The ontology is designed with dual-language support using **multilingual labels** *(rdfs:label)* in both English and Greek. This ensures that every concept, from the author's details to the specific research methodology, is clearly defined and accessible regardless of the user's language. By using these labels, we transform abstract code into a human-friendly format. When visualized as a **graph**, the ontology acts like a 'living map': it connects dissertations with writers, survey methods, and sample sizes, allowing someone to instantly see how different research projects are linked through shared characteristics like academic departments or support dates.

Table 2. *(a) Part A:Research-related fields.(b) Part B: General thesis description (DublinCore-based)* [1].

(a) Research Strategy & Methodology		
	Πεδίο (Field Description)	**Ontology Field**
1	Research Strategy	:strategy
2	Data Production	:sampleProd
3	Time/Space Management	:timeAnalysis, :spaceAnalysis
4	Purpose	:target
5	Research Questions	:studyQuestions
6	Working Hypotheses	:workAssumptions
7	Research Hypotheses	:studyAssumptions
8	Reference Time of Data	:sampleTimeReference
9	Data Collection Time	:sampleCollectionReference
10	Country of Reference	:stateOfReference
11	Geographical Coverage	:spacialCoverage
12	Geographical Unit	:geographicalUnit
13	Population	:population
14	Observation Unit	:basicUnit
15	Sample (Size)	:sampleSize
16	Sampling Method	:collectionMethod
17	Data Recording Method	:methodOfRecording
18	Data Analysis Method	:methodOfAnalysis
19	Recording Tools	:recordingTools
20	Analysis Tools	:analysisTools
(b) Academic Metadata		
1	Author Name	:firstName, :surName
2	Author Gender	:gender
3	Title	:title
4	Supervisor Name	:nameOfSupervisor
5	Defense Date	:dateOfSupport
6	Number of Pages	:numberOfPages
7	Language	:language
8	Abstract	:abstract
9	Keywords	:keyWords
10	Alternative Title	:altTitle
11	Committee Member 1	:nameOfCom1
12	Committee Member 2	:nameOfCom2
13	Additional Notes	:otherNotes

Extra Ontology Data – Object Properties		
+	Internal Relation	:hasWriter,:hasSurvey,: hasSample,:hasSpacetime
+	Institution Name	:institution
+	Department Name	:department
+		:sampleDescription
+		:libraryLink
+		:program
+		:fathersName
+		:typeOfDissertation

Figure 1. *Semantic Ontology in Protege- Class Hierarchy.*

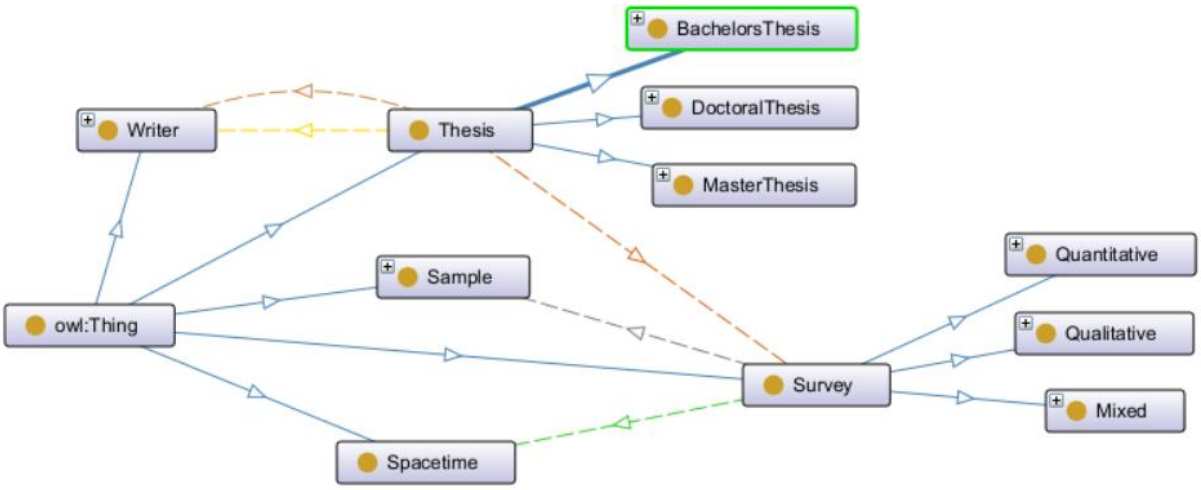


Figure 2. *Semantic Ontology in Protege – Doctoral Thesis.*

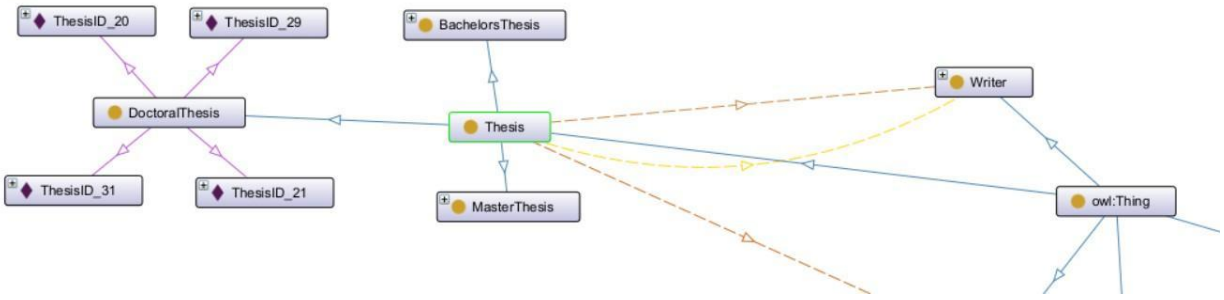


Figure 3. *Semantic Ontology in Protege – Survey details.*

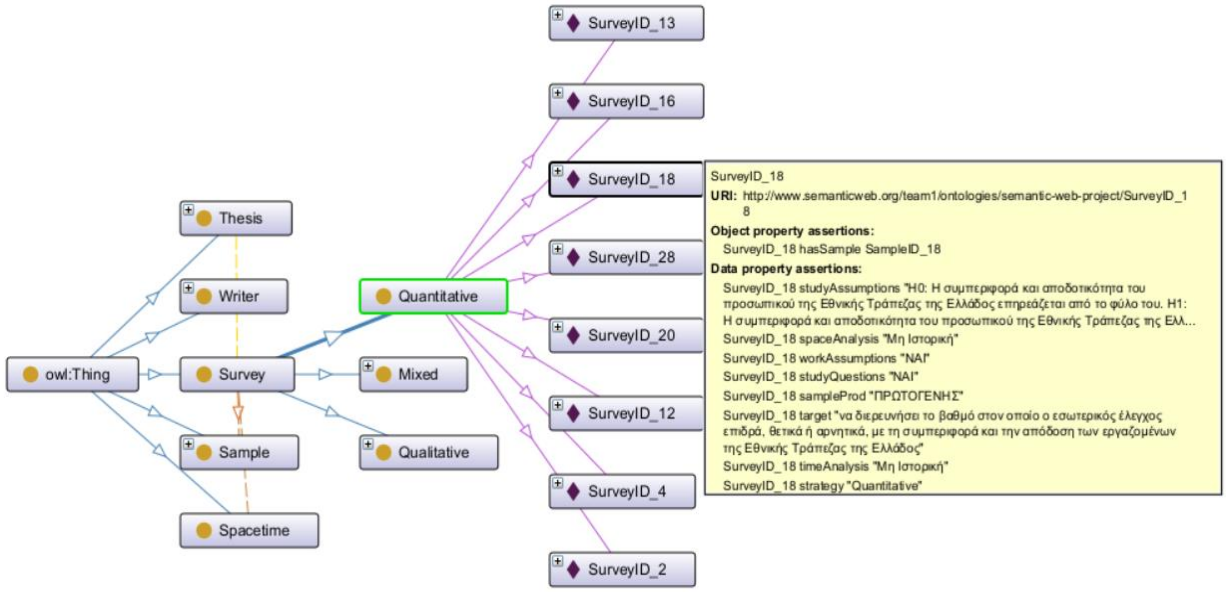


Figure 4. *Semantic Ontology in Protege – Survey visualization.*

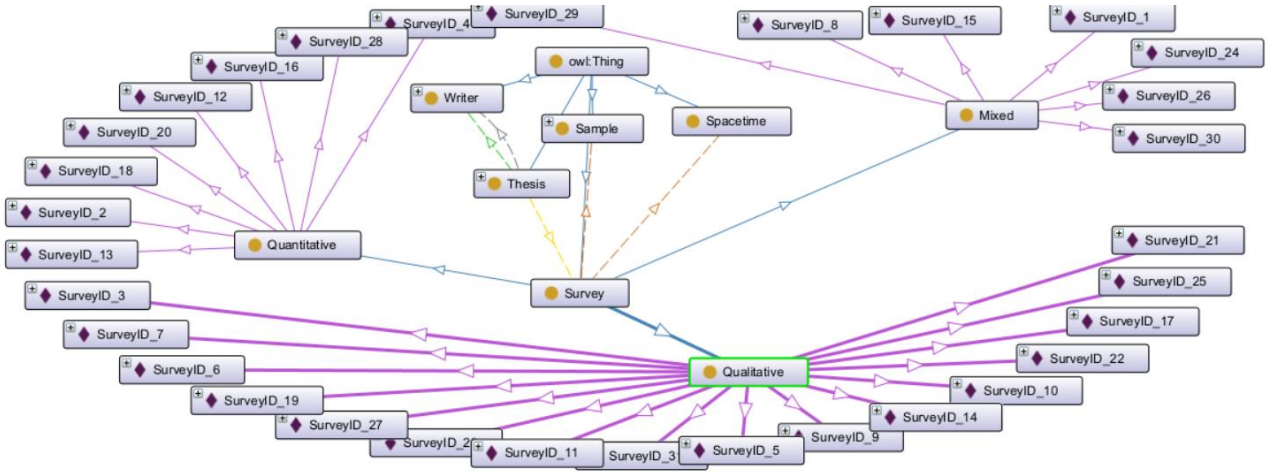
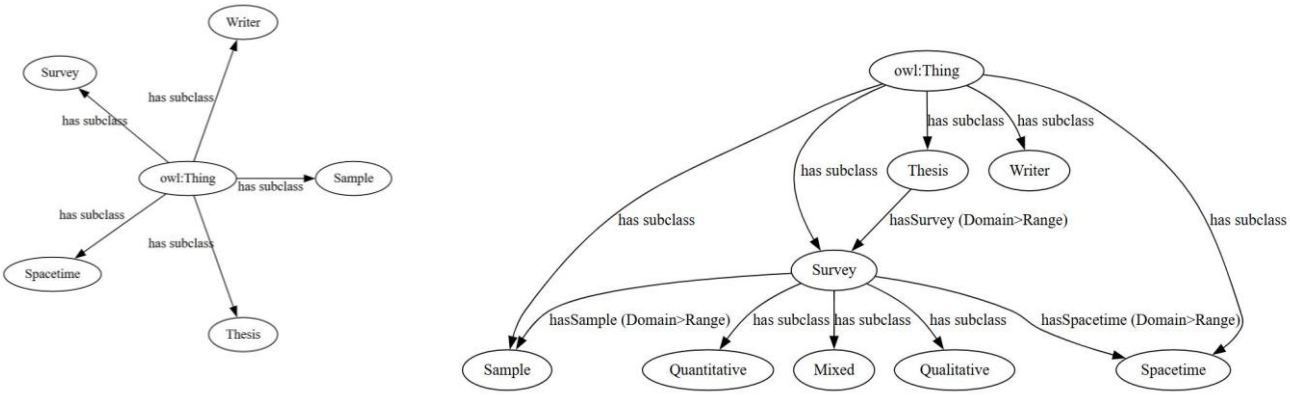
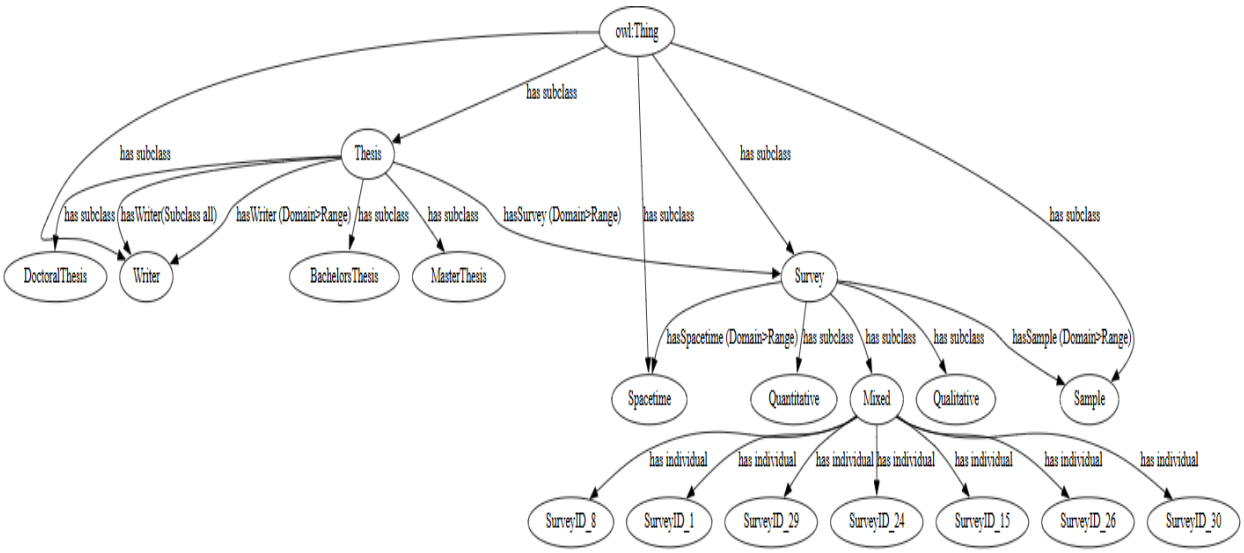


Figure 5. *Graph-based visualization of Semantic Ontology through Graphviz Online.*





2.2 Automated Metadata Extraction

2.2.1 Automated Data Extraction from Theses (Python-LLM Pipeline)

To enrich the ontology that was created, more than 50 doctoral, master's and bachelor's theses were selected to extract the necessary data and match it with the 42 fields of the developed ontology. For the automated and AI-assisted process of extracting data from the theses, a Python program was developed which, communicating with LLM Google Gemini (model *gemini-3-flash-preview*), analyzed the text of the theses (PDF files) and matched this content with the 42 fields, storing the result in structured CSV files, maintaining the accuracy of the original wording so that the information retrieved from the theses could then be evaluated. After evaluating the content of these files, a final selection of 31 theses was made for use in the project.

2.2.2 Conversion to RDF

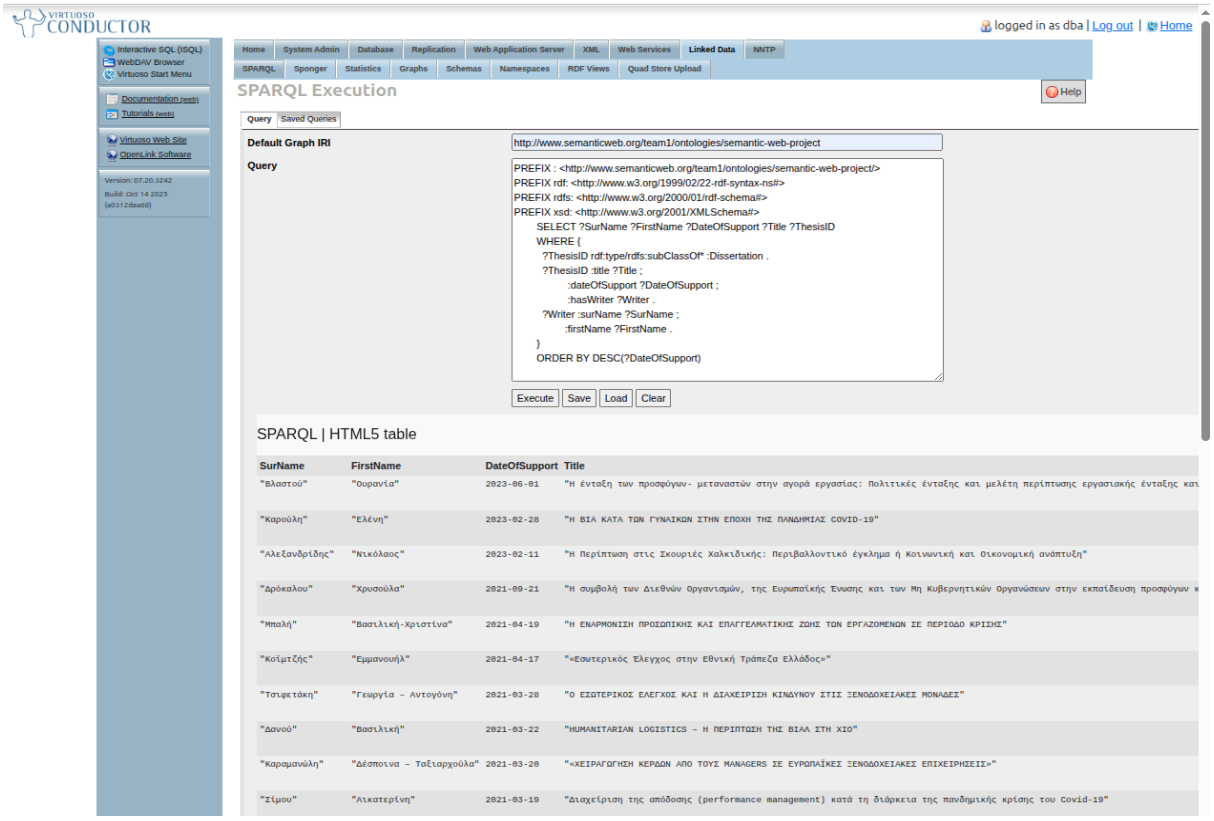
Using an automated process, a Python program was developed to convert the CSV files into Turtle (.ttl) files with the information distributed across the 42 fields of the ontology, as each thesis from the CSV files was mapped to an instance (individual) of the ontology. In this process, unique URIs were created for each thesis (example:*ThesisID_23*) with the necessary triples linking the author, the thesis, and the research.

The Turtle files were initially imported into the Protégé program so that the information extracted could be displayed in the Knowledge Graph, evaluated, and the integrity of the data checked within the ontology.

2.2.3 Database Import of Triples

Once the alignment of the data extracted from the theses with the OWL ontology in Protégé had been ensured, the next step was to import the ontology and triples into Virtuoso Universal Server. Virtuoso allows the ontology and individuals to be hosted, SPARQL queries to be created and submitted, and the data to be displayed. Virtuoso acts as a SPARQL endpoint for executing complex queries on the entire data set. At the same time, it allows connection to external programs so that SPARQL queries can be submitted from there, and the information can be displayed.

Figure 6. *Virtuoso Universal Server*



2.3 Semantic Search and SPARQL Queries

To make our Knowledge Graph truly useful, we developed a set of **SPARQL queries** that allow users to search the repository in ways that were not possible before. While traditional systems only let you search by title or author, our semantic approach allows for "deep" searches based on the actual research methodology. In total, we created 31 different query templates to power our web application. Because our ontology is built with a hierarchical structure, these queries can automatically include all sub-types of a category. For example, a search for a "Survey" will correctly retrieve results labeled as Qualitative, Quantitative, or Mixed-methods research. A summary of the main query types and their roles in the system is presented in the table below:

Table 3. *A Taxonomy of SPARQL Queries for Data Discovery and Analysis.*

Methodology-Based Search:	These queries filter *theses* by their research strategy. For example, by_***strategy: Qualitative*** query retrieves only theses that used qualitative methods like interviews or case studies.
Advanced Greek Text Search:	Since our data includes Greek text, we created a specialized query (search_keyword) that uses a "cleaning" mechanism. It removes accents (tones) from Greek characters so that a user can find results regardless of how they type the search term.
Academic Level Filtering:	Using the hierarchy we built queries like *MasterThesis* or *Bachelors Thesis* to allow users to isolate research based on the specific degree level.
Statistical Analysis:	We also included 'counting' queries, such as *stats_gender* and *stats_years*, which provide a statistical analysis of the repository's demographics based on gender and support years.

By using these queries, our system transforms the "dark data" of the repository into an active, searchable Knowledge Graph where research patterns can be discovered in seconds.

3. Web Application Layer and User Interface Implementation

3.1 Architectural Overview

Among the programming languages and corresponding libraries that can communicate with Virtuoso Universal Server, we chose to use the Python programming language in combination with the Flash Web Framework. The Flask Micro-framework is ideal for developing web applications in Python as it fully implements the MVC (Model View Controller) model and uses a template engine to create dynamic HTML pages. In our case, it was used to submit SPARQL queries to Virtuoso and display the results of our SPARQL queries in elegant tables or graphs. The Python library SPARQLWrapper was used in the web application that was developed. The web application was designed as a thin interaction layer between the end user and the Virtuoso-hosted RDF Knowledge Graph. The objective was to provide structured access to SPARQL-based semantic queries without exposing users to query syntax. The system follows a three-tier architecture:

1. **Presentation Layer** (HTML5, CSS3, JavaScript)
2. **Application Layer** (Flask backend with SPARQLWrapper)
3. **Data Layer** (Virtuoso Triple Store)

The frontend communicates with the backend through HTTP requests. The backend dynamically constructs SPARQL queries using predefined query templates and executes them against the Virtuoso endpoint. Results are returned in JSON format and rendered in the UI using server-side templating (Jinja2). This separation ensures that:

- The ontology remains independent of the UI.
- SPARQL logic is encapsulated in the backend.
- The frontend is responsible solely for visualization and interaction.

3.2 Query Abstraction and Parameterized Interaction

Rather than allowing arbitrary SPARQL input, the interface implements a **controlled query abstraction model**. Each UI filter (e.g., supervisor, methodology, sample size threshold, institutional constraints, temporal or spatial analysis classification) maps to a predefined SPARQL template. User input is passed as parameters to these templates. For example:

- Supervisor-based search injects a variable into a `FILTER` or triple pattern.
- Sample size filtering introduces a numeric constraint using `FILTER (?sampleSize > X)`.
- Institutional filtering conditionally binds `?institution` and optionally `?department`.

This approach:

- Prevents malformed queries.
- Enforces semantic consistency with the ontology.
- Ensures that only meaningful graph patterns are executed.

The frontend does not construct SPARQL syntax directly. Instead, it transmits structured form parameters to Flask routes, which select and parameterize the appropriate query template.

3.3 Dynamic Result Rendering

SPARQL results returned by Virtuoso follow the standard JSON bindings structure:

```
{"results": {

"bindings": [{

"Title": { "type": "literal", "value": "..." },

"Author": { "type": "literal", "value": "..." }

}]}}
```

The backend extracts:

- `results["results"]["bindings"]` → row data
- `results["head"]["vars"]` → column identifiers

These are passed to the template as:

- `Results`
- `columns`

The frontend table is rendered dynamically using Jinja2 iteration over `columns`, ensuring that:

- The table adapts to different SPARQL query outputs.
- No static column definitions are required.
- The UI remains query-agnostic.

Optional SPARQL variables (`OPTIONAL` patterns) are handled gracefully. If a variable is absent from a binding, the UI displays a placeholder (e.g., "—"), preserving structural consistency across rows.

3.4 Modal-Based Record Inspection

To avoid cluttering the main result table while still providing access to full metadata, a client-side modal inspection mechanism was implemented.

Each row includes a “View” button. When triggered:

1. The entire SPARQL binding object is serialized to JSON using Jinja’s `tojson` filter.
2. The serialized object is embedded as a `data-*` attribute in the button.
3. JavaScript parses this JSON payload.
4. The modal dynamically generates key–value cards for each variable.

This design:

- Eliminates the need for additional backend requests.
- Avoids re-executing SPARQL queries.
- Enables query-independent record inspection.

State management is handled entirely client-side. The modal maintains an internal reference to the selected record and supports:

- Escape-key closing
- Overlay-click closing
- Clipboard copying of the thesis URI

This ensures a clean separation between server-rendered data and client-side interaction logic.

3.5 Statistical Visualization Pipeline

The statistics page demonstrates aggregate SPARQL queries (e.g., COUNT, GROUP BY, ORDER BY). The backend executes aggregation queries such as:

- Number of theses per year
- Gender distribution

The resulting arrays are passed to the frontend via a hidden DOM “data bridge” element:

```
<div id="data-bridge"

data-years='[...]'

data-year-counts='[...]'

data-genders='[...]'

data-gender-counts='[...]'>

</div>
```

JavaScript retrieves these values via `dataset` attributes and initializes Chart.js visualizations. This approach avoids additional API endpoints and enables deterministic, server-controlled data injection. Two types of charts are implemented:

- **Bar chart** for temporal distribution
- **Doughnut chart** for categorical gender distribution

Chart configuration includes:

- Responsive scaling
- Custom tooltips
- Thematic styling aligned with the overall UI

3.6 Automated Insight Layer

Beyond raw visualization, an automated insight computation layer was implemented at the frontend level. Using the aggregated arrays, the system computes:

- Peak production year (argmax of yearly counts)
- Average theses per year
- Gender percentage distribution
- Recent trend comparison (last three years vs previous three)

These insights are rendered as structured cards below the charts. Importantly, no additional SPARQL queries are required. All computations are derived from existing aggregate results, demonstrating the extensibility of the visualization layer.

3.7 UI Design and Interaction Model

The interface adopts a modern interaction paradigm characterized by:

- Glassmorphism-inspired layered panels
- Responsive CSS grid layout
- Animated transitions
- Hover elevation effects
- Modal overlays

Although aesthetic in nature, these choices serve functional purposes:

- Visual grouping of ontology-aligned filters
- Improved readability for dense tabular data
- Clear action affordances (e.g., “View”, “Back”, “Print”)
- Reduced cognitive load during result exploration

The UI remains fully decoupled from the RDF schema. Any extension of the ontology or addition of SPARQL variables can be supported without structural changes to the frontend.

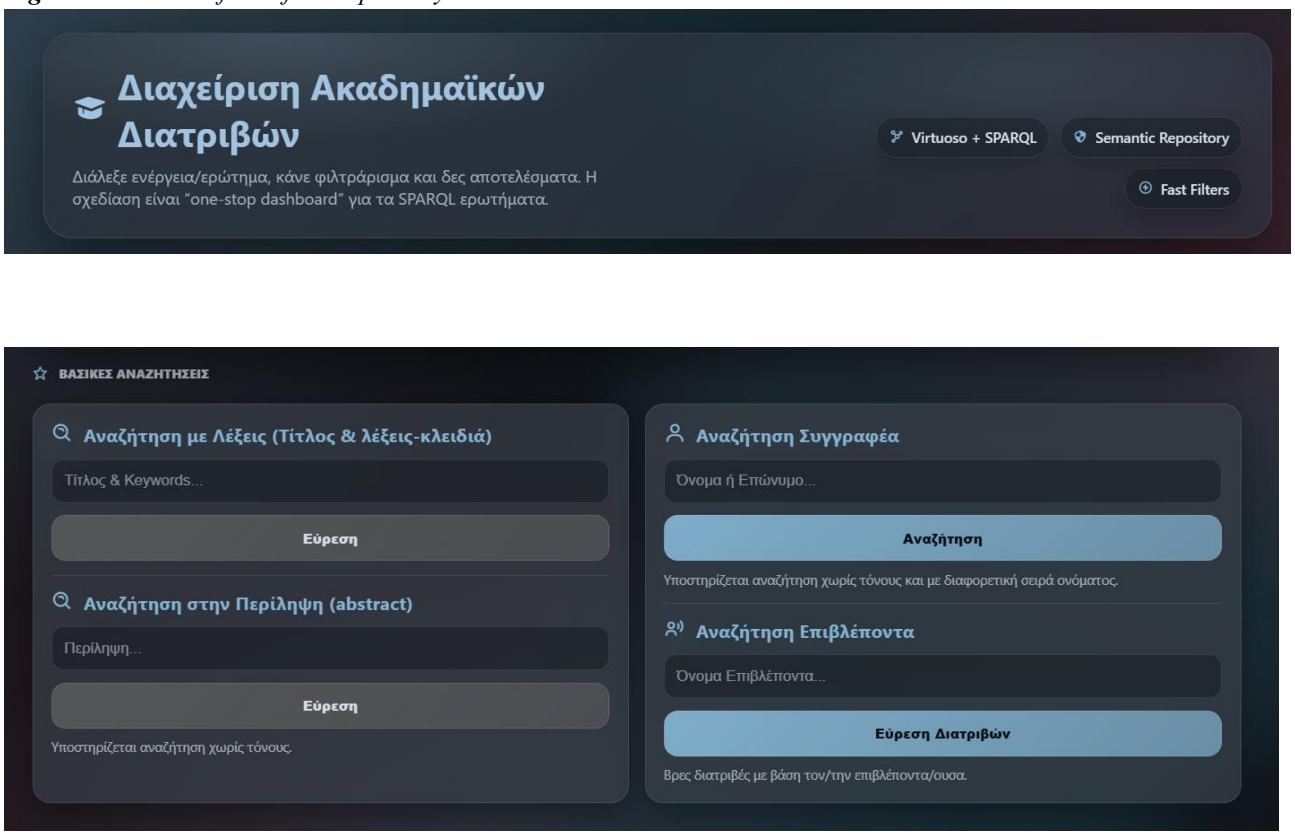
3.8 Contribution to System Usability

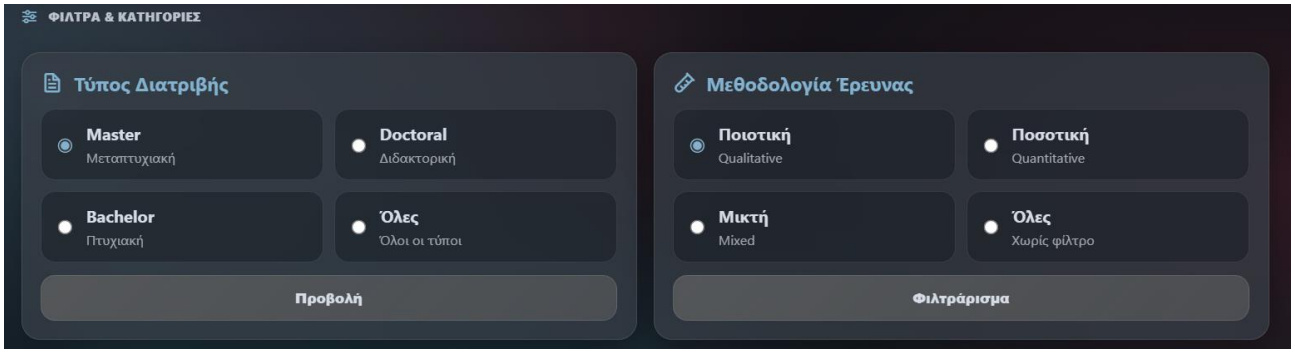
The user interface operationalizes the ontology by:

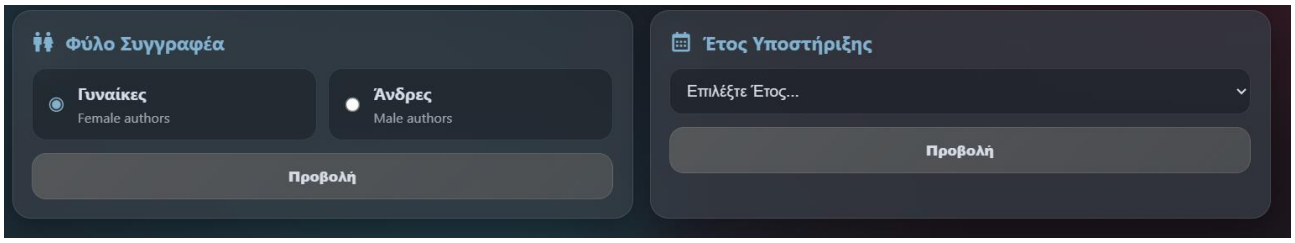
- Translating semantic classes into filterable categories.
- Exposing empirical metadata (methodology, sampling, temporal and spatial dimensions) through structured UI components.
- Transforming RDF triples into interpretable research information.

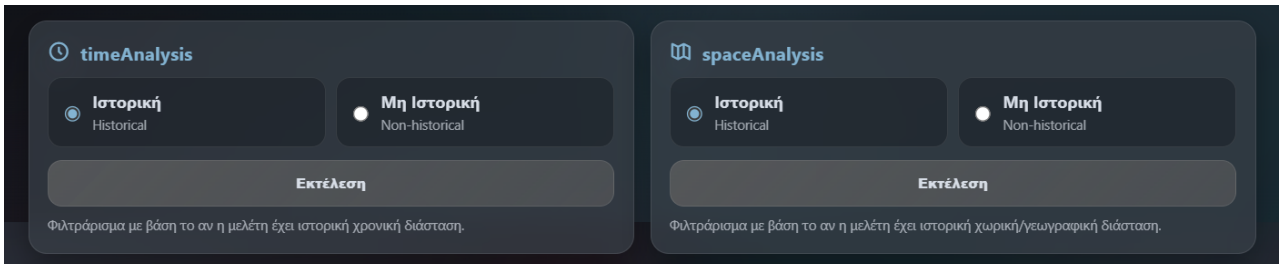
While the ontology defines structure and SPARQL enables retrieval, the web application enables discoverability. Without this interface layer, the Knowledge Graph would remain accessible only to users familiar with SPARQL, limiting its practical utility.

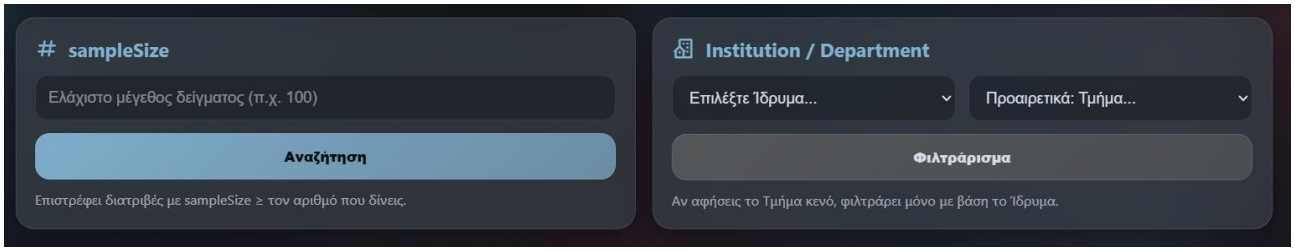
Figure 7: *Web interface of the repository's main dashboard.*




ΦΙΛΤΡΑ & ΚΑΤΗΓΟΡΙΕΣ
Τύπος Διατριβής
Master
Μεταπτυχιακή
Doctoral
Διδακτορική
Bachelor
Πτυχιακή
Όλες
Όλοι οι τύποι
Προβολή
Μεθοδολογία Έρευνας
Ποιοτική
Qualitative
Ποσοτική
Quantitative
Μικτή
Mixed
Όλες
Χωρίς φίλτρο
Φιλτράρισμα


Φύλο Συγγραφέα
Γυναίκες
Female authors
Άνδρες
Male authors
Προβολή
Έτος Υποστήριξης
Επιλέξτε Έτος...
Προβολή


timeAnalysis
Ιστορική
Historical
Μη Ιστορική
Non-historical
Εκτέλεση
Φιλτράρισμα με βάση το αν η μελέτη έχει ιστορική χρονική διάσταση.
spaceAnalysis
Ιστορική
Historical
Μη Ιστορική
Non-historical
Εκτέλεση
Φιλτράρισμα με βάση το αν η μελέτη έχει ιστορική χωρική/γεωγραφική διάσταση.


sampleSize
Ελάχιστο μέγεθος δείγματος (π.χ. 100)
Αναζήτηση
Επιστρέφει διατριβές με sampleSize ≥ τον αριθμό που δίνεις.
Institution / Department
Επιλέξτε Ίδρυμα...
Προαιρετικά: Τμήμα...
Φιλτράρισμα
Αν αφήσεις το Τμήμα κενό, φιλτράρει μόνο με βάση το Ίδρυμα.

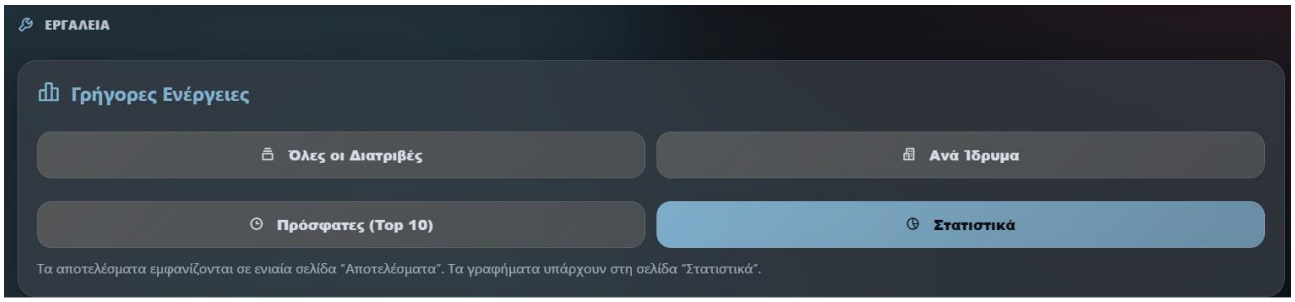

ΕΡΓΑΛΕΙΑ
Γρήγορες Ενέργειες
Όλες οι Διατριβές
Ανά Ίδρυμα
Πρόσφατες (Top 10)
Στατιστικά
Τα αποτελέσματα εμφανίζονται σε ενιαία σελίδα "Αποτελέσματα". Τα γραφήματα υπάρχουν στη σελίδα "Στατιστικά".

Figure 8: *Statistical overview of the repository content.*

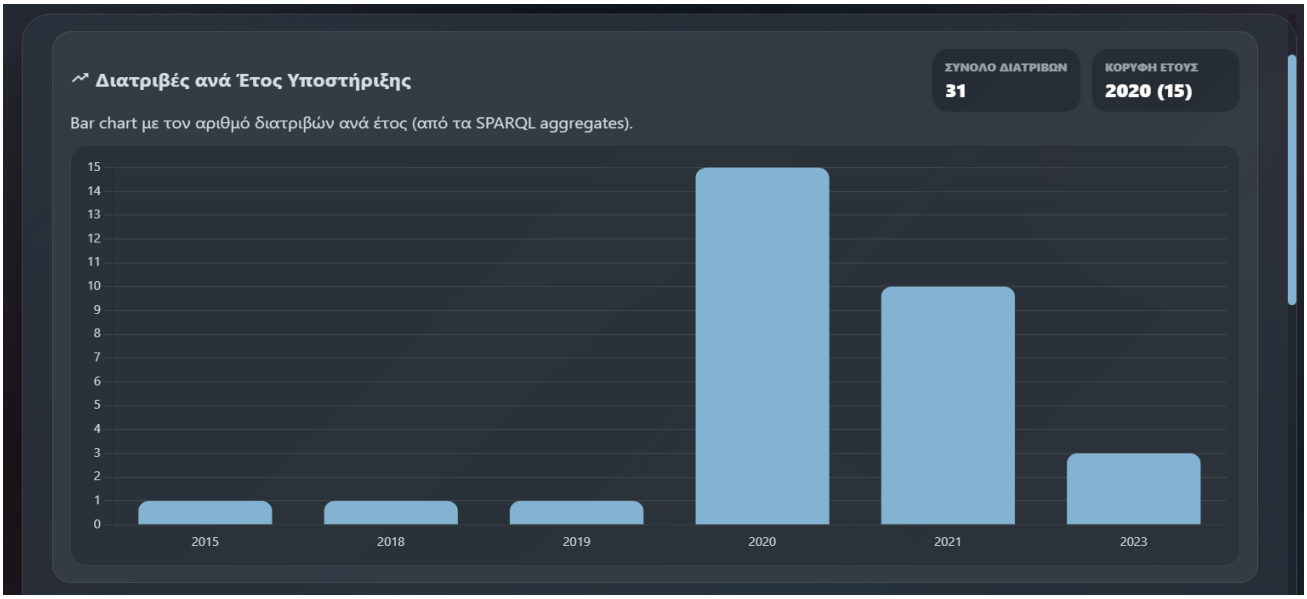
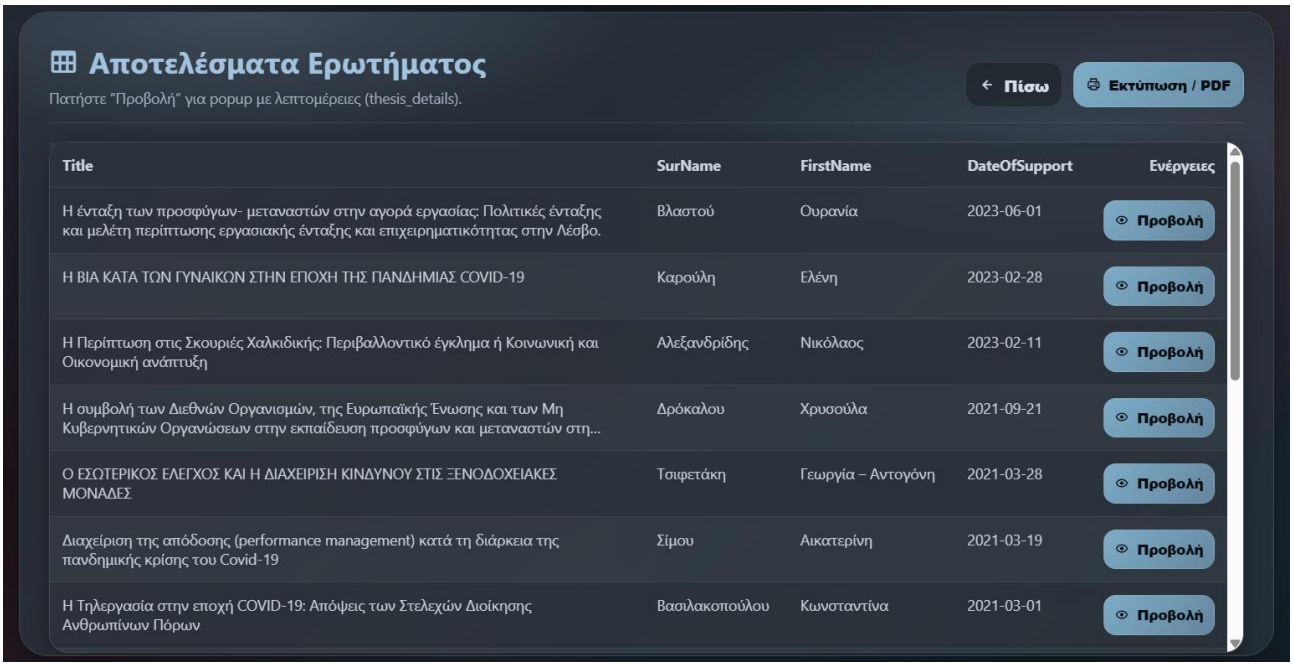


Figure 9: *The search results interface displaying filtered thesis based on user criteria.*



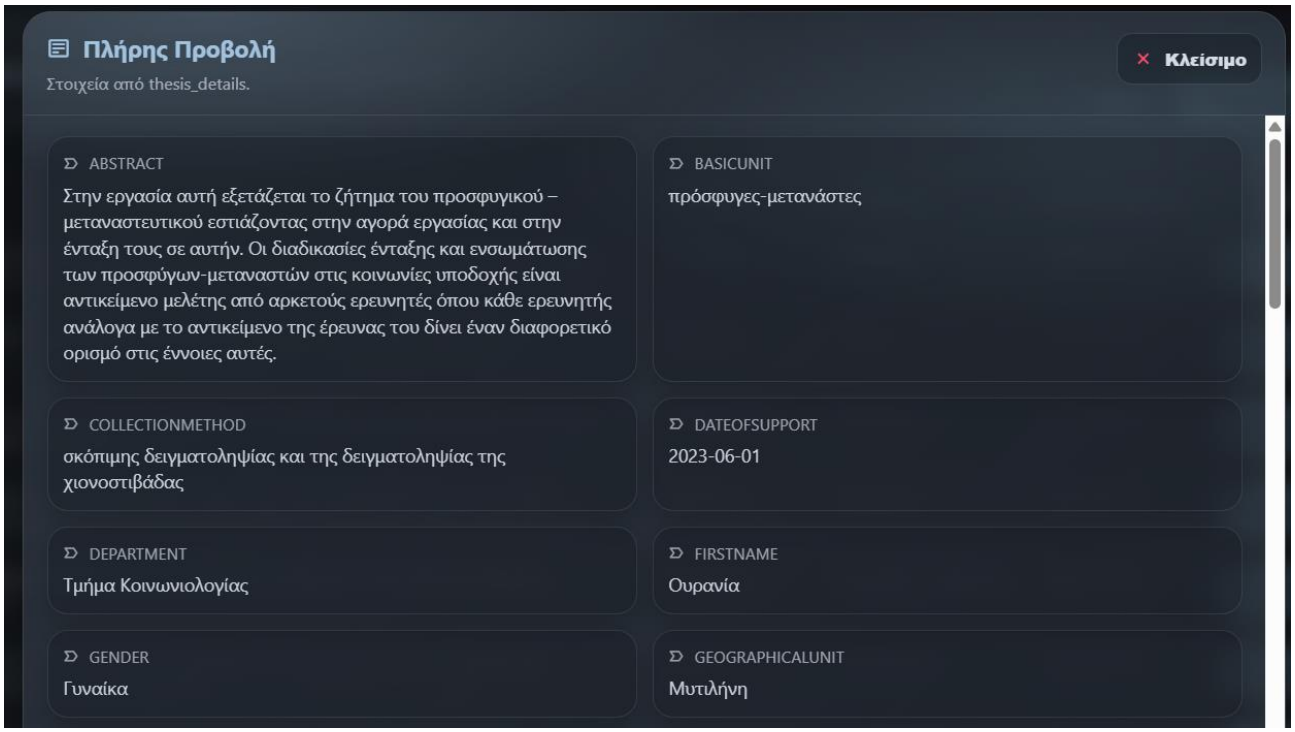
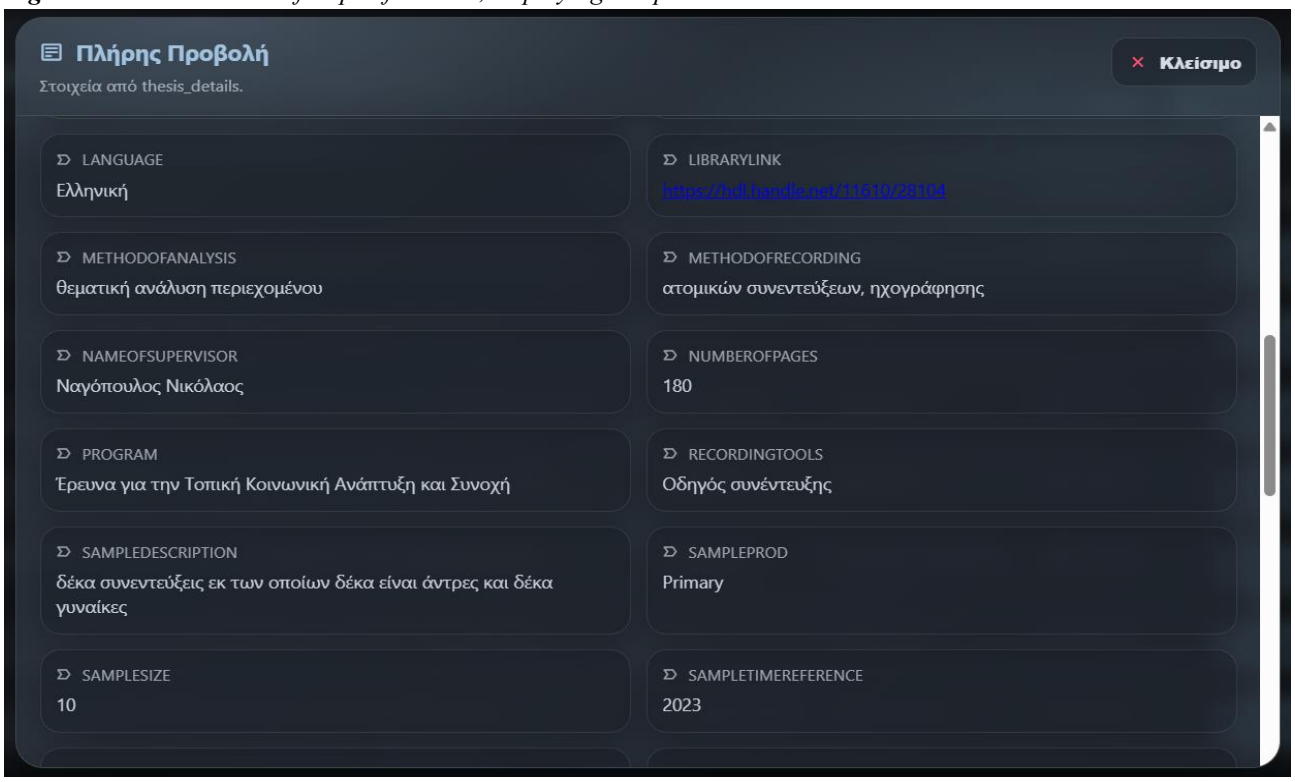


Figure 10: *Detailed view of a specific thesis, displaying comprehensive metadata.*



4. Related Work

Organizing and discovering research within university libraries has remained a significant challenge for decades. Most academic library systems rely on traditional standards like MARC to catalog their collections. While MARC is effective for general library management, it lacks the necessary detail to describe the complex internal structure of a student's thesis, such as the specific data collection methods or the analytical tools used.

Systems like Ellanikos illustrate this gap, they do not currently support the level of metadata specialization needed to align with modern scientific documentation standards. Because traditional library frameworks often overlook the empirical aspects of a study, much of the research's methodological value remains hidden [1]. Furthermore, when cataloging is handled strictly by library staff without the author's input, essential details about how the research was conducted are frequently lost.

The Simplicity versus Complexity Gap: Most institutional repositories today use **Dublin Core (DC)**. It is very popular because it is simple using only 15 basic fields like Title, Creator, and Date. While this simplicity makes it easy to use, it creates a "discovery gap". A researcher cannot search for a thesis based specifically on whether it used "qualitative interviews" or a "sample size of 200," because Dublin Core does not have fields for these details. On the other hand, there are very powerful standards like the **Data Documentation Initiative (DDI)**. DDI can describe every tiny detail of a research study, but it is extremely complex. It was designed for professional data

experts, not for students or busy university staff. As a result, many valuable student theses end up as "dark data", stored in a system but nearly impossible to find through specific research filters.

Moving Toward a Semantic Solution: Recent research [1] has suggested that a **hybrid approach** is the best way forward. Instead of choosing something too simple or too complex, we combined the user-friendly structure of **Dublin Core** with the specific research details found in **DDI**. This allowed us to create a "lightweight" model that is easy for anyone to fill out but still stays packed with all the rich information a researcher needs. Furthermore, using **Semantic Web** technologies like **OWL** and **RDF** allows this metadata to move beyond simple text lists. It turns the repository into a **Knowledge Graph**, where different research components are linked together. This is the foundation of our project: taking the best parts of existing standards to make student research visible and globally compatible.

5. Discussion

The development of this ontology and the subsequent Knowledge Graph demonstrate a practical way to solve the "dark data" problem in academic repositories. By moving from a **core XML schema** to a structured **OWL ontology**, we have created a system that understands the context of a thesis, not just its text.

Semantic Compatibility: A key achievement of this project was the successful mapping of local fields to global vocabularies like **Dublin Core** and **Schema.org** . In our ontology, we used properties such as *rdfs:seeAlso* and *rdfs:isDefinedBy* to link our specific data (like *collectionMethod* or *methodOfAnalaysis*) to international standards. This ensures that our metadata is **globally compatible**, allowing other semantic systems to understand our data without manual translation.

Impact on the Gray Literature: By assigning structured metadata to student theses, we elevate the status of the **gray literature**. These works are no longer "hidden" files, they become **Linked Open Data**. This makes the empirical evidence produced by students reusable for secondary analysis, potentially saving time and resources for future researchers who can now find existing data samples and methodologies with ease.

6. Conclusions

The main goal of this project was to prove that student research does not have to remain "dark data" hidden within institutional repositories. By transforming a **core XML schema** into a formal **OWL ontology**, we have demonstrated how Semantic Web technologies can bring transparency and discoverability to the gray literature of higher education.

Our work successfully mapped 33 metadata fields to international vocabularies like **Dublin Core** and **Schema.org**, ensuring that the data is not only machine-readable but also globally compatible. Through the construction of a **Knowledge Graph** and the deployment of a **SPARQL-**

based web application, we showed that it is possible to perform deep, methodological searches that traditional systems simply cannot handle.

In conclusion, this framework offers a scalable solution for universities to elevate the visibility of student theses. By making empirical research findable and reusable, we turn individual academic projects into a collective, accessible body of knowledge that can support future researchers and the broader academic community.

References

1. G. Vassiliou, G. Tsamis, S. Chatzinikolaou, T. Nipurakis, and N. Papadakis, "Enhancing Discoverability: A Metadata Framework for Empirical Research in Theses," *Algorithms*, vol. 18, no. 8, p. 490, Aug. 2025. Available: https://doi.org/10.3390/a18080490.
2. S. Nicholson and T. Bennett, "Do institutional repository deposit guidelines deter data discovery?," *Evid. Based Libr. Inf. Pract.*, vol. 16, no. 3, pp. 2–17, 2021. Available: https://doi.org/10.18438/eblip29913
3. *The Dublin Core Metadata Element Set*, Standard ANSI/NISO Z39.85-2017, NISO Press, Baltimore, MD, USA, 2017.
4. Dublin Core, "Detailed Information About the Dublin Core, Its Activities and Metadata Sets," 2001. Available: http://www.dublincore.org.
5. DDI Alliance, "The Data Documentation Initiative Description," DDI Alliance, Ann Arbor, MI, USA, 2017. Available: https://ddialliance.org/.
6. N. F. Mosha and P. Ngulube, "Metadata Standard for Continuous Preservation, Discovery, and Reuse of Research Data in Repositories by Higher Education Institutions: A Systematic Review," Information, vol. 14, no. 8, p. 427, 2023. Available:https://doi.org/10.3390/info14080427
7. M. Vardigan, "Data Documentation Initiative: Toward a Standard for the Social Sciences," *Int. J. Digit. Curation*, vol. 3, no. 1, pp. 107–113, 2007. Available: https://doi.org/10.2218/ijdc.v3i1.45
8. Schema.org, "Schema.org Vocabulary," 2025. Available: https://schema.org/.
9. W3C, "RDF 1.1 Primer," 2014. Available: https://www.w3.org/TR/rdf11-primer/.
10. OpenLink Software, "Virtuoso Universal Server: Open source edition," 2026. Available: https://virtuoso.openlinksw.com/
11. W3C, "SPARQL 1.1 Query Language," W3C Recommendation, Mar. 2013. Available: https://www.w3.org/TR/sparql11-query/.
12. Protégé Project, "Protégé Desktop User Documentation," Stanford Center for Biomedical Informatics Research, 2026. Available: https://protege.stanford.edu/support.php
13. Pallets, “Flask Documentation (2.3.x),” Flask.palletprojects.com. https://flask.readthedocs.io/ .